

Assignment 05

**הרצת מודל שפה גדול באופן מקומי:
AirLLM, קוונטיזציה והשוואת ביצועים**

**EX05 - Running a Massive LLM Locally: AirLLM,
Quantization and Performance Benchmarking**

מטלה 05 | שיעור L08

ד"ר יורם סגל

כל הזכויות שמורות לד"ר יורם סגל ©

June 2026

גרסה 1.0

מילות מפתח: AirLLM, קוונטיזציה, Quantization, LoRA, Ollama, Hugging Face, הסקה מקומית, On-Premises LLM, CPU מול GPU, VRAM, Prefill, Decode, Latency, Throughput, זיכרון וירטואלי, Virtual Memory, Paging, Benchmarking, SafeTensors, GGUF, מדדי ביצועים, דוח טכני.

הערה: מסמך זה נכתב בלשון זכר מטעמי נוחות בלבד, אך מיועד לכל המגזרים כאחד.

תוכן העניינים

1	תיאור המטלה: Assignment Overview	3
2	המשימה המרכזית – דוח טכני מעמיק: The Core Mission	3
3	מטרות העבודה: Objectives	4
4	שאלות מחקר והבנה: Research Questions	5
5	משימות הליבה: Core Tasks	5
5.1	א. תיעוד החומרה והצדקת בחירת המודל	5
5.2	ב. קו בסיס – הרצה ישירה (Baseline)	5
5.3	ג. שילוב AirLLM וקוונטיזציה	6
5.4	ד. מדידה והשוואת ביצועים	6
5.5	ה. ניתוח כדאיות כלכלית: On-Prem מול API	6
5.6	ו. ניתוח מושגי ההרצאה	7
5.7	ז. הרחבות ויוזמות מקוריות	7
6	שיקולי תכנון ועבודה יעילה: Planning & Efficiency	8
6.1	עשה: Do	8
6.2	אל תעשה: Don't	9
7	תוצרי ההגשה: Deliverables	9
8	דרישות מפורטות ל-README: README Requirements	10
9	מבנה מומלץ למאגר: Recommended Repository Structure	10
10	ציפיות מן העבודה: Expectations	11
11	נספח עזר: הערכת זמנים ריאלית בשיטת Vibe Coding: Ap-	11
11	pendix – Realistic Time Estimation	11

11	שלב 1: התקנות, הגדרת הסביבה והורדת המודלים .	11.1
12	שלב 2: הרצות, ניסויים ומדידות	11.2
12	שלב 3: עיבוד נתונים, השוואת ביצועים וניתוח כלכלי	11.3
12	שלב 4: חיבור וכתובת הדוח הטכני (README) . . .	11.4
13	סיכום הערכת הזמנים	11.5

1 תיאור המטלה: Assignment Overview

במטלה זו תתבקשו להוכיח, באופן מעשי ומנומק, שאתם שולטים במסלול המלא של הרצת מודל שפה גדול באופן מקומי (On-Premises). הרעיון המרכזי הוא לקחת מודל אחד שגדול **מדי** עבור החומרה שלכם, להראות מדוע הוא נכשל או איטי בלתי-נסבל בהרצה ישירה, ולאחר מכן להפעיל טכניקות אופטימיזציה – AirLLM וקוונטיזציה – כדי לגרום לאותו מודל לרוץ בכל זאת, ולנתח לעומק את המחיר והתועלת של כל גישה.

זוהי מטלה פתוחה ומבוססת חקירה. ההנחיות שלהלן הן **קו מנחה כללי ולא מתכון מדויק**: עליכם לתכנן את הניסוי בעצמכם, לבחור את המודל ואת החומרה, ולהצדיק כל החלטה. מטרת העבודה אינה איכות הפלט של המודל, אלא הבנת מנגנוני ההרצה, הסקת המסקנות, והיכולת להציג ניתוח הנדסי וכלכלי מעמיק.

העבודה תוגש כמאגר GitHub מלא הכולל קוד, ניסויים, ודוח טכני מקיף.

שיקול חשוב – היקף הניסוי ובחירת גודל המודל

מטרת המטלה **אינה** להריץ מודלים במשך שעות או ימים. עליכם לנתח את כוח החישוב של המחשב האישי שלכם, ובהתאם לכך לבחור מודל שיהיה גדול – **אך לא גדול מדי**. מודל כזה אמור לאפשר להדגים השהיה ארוכה (Latency) או חוסר יכולת להריץ ישירות, אל מול השיפור והקלת העומס בעזרת AirLLM.

זכרו שזהו **ניסוי**: אם לא הצלחתם לשפר את הביצועים, מותר ואף רצוי להציג **תוצאה שלילית**, ולהסביר ולנמק מדוע הדברים התנהגו כך במקרה שלכם. תוצאה שלילית מנותחת היטב שווה לא פחות מתוצאה חיובית.

2 המשימה המרכזית – דוח טכני מעמיק: The Core Mission

לב המטלה הוא דוח טכני מעמיק (deep-dive technical report) המתעד את הניסוי שלכם. להלן הגדרת המשימה המרכזית, כפי שעליכם לממש אותה:

כתבו דוח טכני מקיף ומעמיק (deep-dive) המתעד את הניסוי המעשי שלכם בהרצת מודל שפה גדול (LLM) באופן מקומי. התחילו בתייעוד המפרט המדויק של חומרת המחשב שלכם (GPU/CPU) כדי להצדיק את בחירת המודל, ותעדו בצורה חיה את התוצאות – ובמיוחד את צווארי הבקבוק הבלתי-נמנעים בביצועים – של ניסיון להריץ אותו ישירות על המעבד שלכם. לאחר מכן שלבו לתוך הצגת את AirLLM ומנגנון קוונטיזציה, הריצו את הניסוי שוב, והראו כיצד טכניקות אופטימיזציה אלו משנות מן היסוד את הקצאת המשאבים. הנרטיב הסופי חייב לחרוג מעבר לנתונים גולמיים ולנתח לעומק את מושגי ההרצאה באמצעות מדדים השוואתיים, גרפים ממחישים, ותכנון ניסויי מקורי – המוכיחים הבנה יסודית של פריסת מודלים מקומית (On-Premises LLM Deploy-ment).

3 מטרות העבודה: Objectives

העבודה צריכה להראות יכולת לתכנן ולבצע ניסוי הנדסי שלם, למדוד אותו, ולנתח אותו הן מן ההיבט הטכני והן מן ההיבט הכלכלי. עליכם להתאים מודל לחומרה, לזהות את צוואר הבקבוק האמיתי (חישוב מול זיכרון), להפעיל טכניקות אופטימיזציה בצורה מבוקרת, ולכמת את ההשפעה שלהן.

נדרש להציג לא רק מה קרה, אלא גם **מדוע** – לקשור כל תוצאה למושגי ההרצאה (Decode/Prefill, VRAM, זיכרון וירטואלי, קוונטיזציה), ולהסיק מתי כל גישה כדאית.

על העבודה להפגין שימוש במדדי ביצוע סטנדרטיים בתעשייה. מטרת-העל היא להוכיח בעזרת נתונים – ולא בהשערות בלבד – האם המודל והחומרה שבחרתם חסומים בכוח חישוב (compute-bound) בעת קריאת הקלט, או חסומים ברוחב פס הזיכרון (memory-bound) בעת ייצור התשובה. שאיפה מתקדמת היא לשרטט "מודל גג" (Roofline Model) – ייצוג ויזואלי הממחיש מתי המערכת עוברת ממגבלת משאב אחת לאחרת.

4 שאלות מחקר והבנה: Research Questions

- במהלך העבודה יש להתייחס במפורש לשאלות הבאות כחלק מן הדוח:
- מהו צוואר הבקבוק שמנע את ההרצה הישירה – זיכרון (RAM/VRAM) או כוח חישוב? כיצד זיהיתם זאת?
 - כיצד AirLLM משנה את הקצאת המשאבים, ומהו הקשר לזיכרון הווירטואלי ולמנגנון ה-Paging ממערכות הפעלה?
 - מה הייתה ההשפעה של הקוונטיזציה על צריכת הזיכרון, על המהירות, ועל איכות הפלט? היכן עבר "קו האדום" של הדיוק?
 - כיצד באים לידי ביטוי שלבי ה-Prefill וה-Decode במדידות שלכם, וכיצד הם משתקפים בהפרדה שבין מדד ה-TTFT (Time To First Token), המייצג את קצב החישוב, לבין מדד ה-TPOT (Time Per Output Token), המייצג את עומס הזיכרון?
 - מהו המחיר (Throughput/Latency) שאתם משלמים עבור היכולת להריץ מודל גדול על חומרה צנועה?
 - מתי כדאי כלכלית לעבוד מקומית, ומתי עדיף להשתמש ב-API חיצוני?

5 משימות הליבה: Core Tasks

5.1 א. תיעוד החומרה והצדקת בחירת המודל

תעדו את המפרט המדויק של המחשב שלכם: דגם ה-CPU, מספר הליבות, גודל ה-RAM, דגם ה-GPU (אם קיים) וגודל ה-VRAM, וכן סוג האחסון (NVMe/SSD). על בסיס מפרט זה, בחרו מודל מ-Hugging Face **שגודל מכדי** להיכנס בנוחות לחומרה שלכם, והסבירו את ההיגיון מאחורי הבחירה (מספר פרמטרים, פורמט, גודל).

5.2 ב. קו בסיס – הרצה ישירה (Baseline)

נסו להריץ את המודל הנבחר ישירות על החומרה שלכם (למשל דרך Ollama או Hugging Face). תעדו בצורה חיה את מה שקורה: האם המודל נכשל בטעינה, נתקע, או רץ באיטיות בלתי-נסבלת? זהו את צוואר הבקבוק והסבירו אותו. זהו ה-baseline שאליו תשוו את שאר הניסויים.

5.3 ג. שילוב AirLLM וקוונטיזציה

הכניסו לצנרת שלכם את AirLLM ומנגנון קוונטיזציה, והריצו את אותה משימה בדיוק. הראו כיצד הטכניקות הללו משנות מן היסוד את הקצאת המשאבים ומאפשרות למודל לרוץ. תעדו אילו רמות קוונטיזציה ניסיתם (למשל FP16, Q4, Q8) ומה הייתה ההשפעה של כל אחת.

5.4 ד. מדידה והשוואת ביצועים

מדדו באופן שיטתי לכל תרחיש לפחות את המדדים הבאים, והציגו אותם בטבלאות ובגרפים:

- **TTFT** (Time To First Token) – הזמן מקבלת הבקשה ועד הפלט של התו הראשון; מדד לעומס שלב ה-Prefill (בניית ה-KV Cache ועוצמת החישוב).

- **TPOT** (Time Per Output Token) / **ITL** (Inter-Token Latency) – קצב זרימת הטוקנים לאחר הטוקן הראשון (מילי-שניות לטוקן); מדד לעומס שלב ה-Decode ולתנועת המידע מן הזיכרון.

- תפוקה כוללת (Throughput) במונחי tokens/sec.

- צריכת זיכרון שיא (RAM ו-VRAM).

- זמן ריצה כולל וצריכת חשמל משוערת.

- הערכה איכותית של איכות הפלט בכל רמת קוונטיזציה.

5.5 ה. ניתוח כדאיות כלכלית: On-Prem מול API

זוהי דרישה כללית ומחייבת. עליכם לבצע **חישוב עלויות** ולהשוות את ביצוע **אותה משימה בדיוק** בשתי דרכים, כדי להראות מתי כדאי כלכלית לעבוד מקומית (On-Premises) ומתי עדיף להשתמש ב-API חיצוני של ספק צד-שלישי:

1. **עלות API של צד-שלישי** – חשבו את מספר הטוקנים (קלט + פלט) לכל בקשה, והכפילו במחיר לטוקן של ספק לבחירתכם (למשל OpenAI, Claude וכדומה). הציגו את העלות לבקשה בודדת ואת העלות לכמות בקשות נתונה.

2. **עלות עבודה מקומית (On-Prem)** – חשבו את עלות החומרה (CAPEX) המופחתת על פני זמן, בתוספת עלות החשמל והתחזוקה (OPEX). גזרו ממנה עלות אפקטיבית לבקשה כפונקציה של נפח השימוש.

על בסיס שני החישובים, מצאו את **נקודת האיזון** (break-even): מאיזה נפח שימוש (מספר בקשות או טוקנים בחודש) העבודה המקומית הופכת לזולה יותר מן ה-API. הציגו זאת בגרף של **עלות מצטברת מול נפח שימוש**, ונסחו המלצה מנומקת: באילו תרחישים עדיף API, ובאילו עדיף On-Prem (כולל שיקולי פרטיות ואבטחת מידע, לא רק עלות).

בחישוב עלות ה-API כדאי להכיר תופעת תמחור עדכנית המשנה את התמונה: Prompt/Context Caching. ספקים המבוססים על ארכיטקטורת Page-dAttention שומרים בזיכרונם את החלקים הקבועים של הפרומפט המערכתי ללא חישוב Prefill חוזר, ולכן מציעים תעריף מוזל משמעותית על אותם טוקנים מוכמנים. בתרחישים של שאלות חזרתיות על מסמך ארוך, מנגנון זה עשוי להזיז את נקודת האיזון – שקלו זאת בניתוח שלכם.

אופציונלי – אפשרות שלישית: ענן GPU

הרוצים בכך יכולים להוסיף אופציה שלישית להשוואה: עלות ביצוע אותה משימה על כרטיס GPU מושכר אצל ספק ענן (Cloud GPU – למשל מחיר לשעת GPU כפול זמן הריצה). שלבו אותה באותו גרף נקודת-האיזון לצד API ו-On-Prem.

יש לציין במפורש את כל ההנחות שביסוד החישוב (מחירים, נפח שימוש, אורך חיי החומרה, תעריף חשמל) כדי שהניתוח יהיה שקוף וניתן לשחזור.

5.6 ו. ניתוח מושגי ההרצאה

קשרו כל ממצא למושגי ההרצאה. הסבירו את התוצאות דרך Prefill מול De-code, compute-bound מול memory-bound, את תפקיד ה-VRAM, ואת האנלוגיה לזיכרון הווירטואלי ולמנגנון ה-Paging שעליו מבוסס AirLLM.

5.7 ז. הרחבות ויוזמות מקוריות

המטלה היא קו מנחה בלבד. עליכם להציע לפחות הרחבה אחת מקורית: תכנון ניסוי נוסף, מדד חדש, גרף השוואתי מעניין, או שילוב טכניקה נוספת (למשל QLoRA/LoRA לאימון, או השוואה בין מספר מודלים בגדלים שונים).

6 שיקולי תכנון ועבודה יעילה: Planning & Efficiency

מטרת הסעיף הזה היא לעזור לכם לבצע את המטלה בזמן סביר ובלי לבזבז משאבים. העיקרון החשוב ביותר הוא להתחיל קטן, לוודא שהצנרת עובדת, ורק אז להגדיל.

6.1 עשה: Do

- צרו סביבה וירטואלית מבודדת (virtual environment); מומלץ uv.
- בדקו את גרסת ה-Python שלכם – אל תשתמשו בגרסה החדשה ביותר, שכן חבילות רבות עדיין אינן מותאמות לה.
- התחילו ממודל קטן וברמת קוונטיזציה אגרסיבית (למשל Q2) רק כדי לוודא ש"הצנרת" עובדת, לפני שעוברים למודל הגדול.
- הגדירו מספר טוקנים מרבי נמוך בבדיקות הראשוניות.
- ודאו מספיק מקום פנוי בדיסק לפני הורדת מודלים גדולים.
- נהלו נכון את שמירת השכבות של AirLLM: פירוק מודל כבד יוצר קובצי SafeTensors רבים ועתירי Disk I/O. הגדירו במפורש את הנתבי `layer_shards_saving_path` כדי לנתב את המטמון לכונן מהיר או למחיצה ייעודית, ובכך להימנע מהצפת כונן מערכת ההפעלה (כונן C:) במהלך הניסוי.
- בעבודה עם מודלים ממשפחת Qwen ודומיהם, השתמשו בפונקציה הכללית `AutoModel` כדי שהמערכת תתאים את עצמה למחלקה הייעודית ותימנע משגיאות חוסר-תאימות (Class mismatch) באתחול.
- מדדו באופן עקבי ושמרו את כל המספרים הגולמיים לצורך הגרפים.

6.2 אל תעשה: Don't

- אל תבחרו מודל ענק שאין שום סיכוי להריץ אפילו עם AirLLM.
- אל תשמרו את ה-Token של Hugging Face בצורה גלויה בקוד.
- אל תסתפקו בנתונים גולמיים בלי ניתוח, גרפים, וקישור למושגי ההרצאה.
- אל תתעלמו מן ההיבט הכלכלי – ניתוח העלות הוא חלק מהותי מן המטלה.
- אל תהפכו את המטלה לפרויקט גמר; עדיף ניסוי ממוקד, נקי, ומנותח היטב.

7 תוצרי ההגשה: Deliverables

- ההגשה תהיה מאגר GitHub מלא, הכולל לכל הפחות:
- קוד מלא של הניסוי וסקריפטים להרצה ולמדידה.
 - הדוח הטכני המעמיק, כולל תיעוד החומרה, ה-baseline, וניסויי ה-AirLLM והקוונטיזציה. **הדוח חייב לשמש כקובץ ה-README של פרויקט ה-GitHub**, וכל הגרפים, הטבלאות וצילומי המסך חייבים להיות משובצים בתוך ה-README עצמו.
 - טבלאות וגרפים השוואתיים של מדדי הביצועים.
 - ניתוח הכדאיות הכלכלית (On-Prem מול API, ואופציונלית ענן GPU), כולל גרף נקודת-האיזון וכל ההנחות.
 - ניתוח הקושר את התוצאות למושגי ההרצאה.
 - תיעוד ההרחבות והרעיונות המקוריים.

8 דרישות מפורטות ל-README: README Requirements

קובץ README.md הוא חלק מהותי מן ההגשה ועליו להיות ברור וקריא לקורא חיצוני. ה-README חייב לכלול:

- מפרט החומרה והנמקת בחירת המודל.
 - תיאור הניסוי, שלביו, וכלי המדידה.
 - סיכום הממצאים: baseline מול AirLLM וקוונטיזציה.
 - סיכום ניתוח הכדאיות הכלכלית והמלצה מנומקת.
 - הסבר הקושר את התוצאות למושגי ההרצאה.
 - הוראות הרצה ברורות לשחזור הניסוי.
- בנוסף, ה-README חייב לכלול אלמנטים ויזואליים: גרפים, טבלאות השוואה, וצילומי מסך התומכים בניתוח ובהצגת התהליך.

9 מבנה מומלץ למאגר: Recommended Repository Structure

מבנה אפשרי למאגר ההגשה:

```
README.md
├── requirements.txt
├── pyproject.toml
├── src/
├── experiments/
├── results/
├── reports/
└── figures/
```

אפשר להתאים את המבנה לצורכי הפרויקט, אך עליו להישאר עקבי, ברור, וקל לניווט.

10 ציפיות מן העבודה: Expectations

העבודה המצופה היא ניסוי הנדסי מלא ומנותח, המדגים הבנה אמיתית של הרצת מודלים גדולים באופן מקומי. נדרש להראות יכולת להתאים מודל לחומרה, לזהות צווארי בקבוק, להפעיל טכניקות אופטימיזציה, למדוד בצורה שיטתית, ולנתח את התוצאות הן טכנית והן כלכלית.

מטלה זו אינה מיועדת לביצוע מכני של צעדים טכניים בלבד. היא נועדה לבחון כיצד אתם מתכננים ניסוי, כיצד אתם מודדים ומציגים נתונים, וכיצד אתם מסיקים – על בסיס מדדים וחישובי עלות – מתי פריסת מודל מקומית עדיפה, ומתי עבודה מול API חיצוני היא הבחירה הנכונה.

11 נספח עזר: הערכת זמנים ריאלית בשיטת Vibe Coding: Ap- pendix – Realistic Time Estimation

מאחר שאתם מבצעים מטלה זו תוך שימוש בפרדיגמת Vibe Coding (אורקסטרציה של סוכני AI מבוססי מודלי שפה גדולים לכתיבת הקוד והרצת הניסויים), חוויית הפיתוח וניהול הזמן שונים לחלוטין מכתיבת קוד מסורתית. במטלה זו אתם מתפקדים כארכיטקטים וכמנהלי מעבדה, בעוד סוכני ה-AI הם אלו ש"מלכלכים את הידיים" ביישום בפועל.

חשוב להבין שבעוד שזמן הקידוד, כתיבת הסקריפטים והדיבוג מתכווץ דרמטית הודות למכפילי הכוח של מודלי ה-LLM, מגבלות החומרה הפיזיות נותרות בעינן. תהליכי הורדה מהרשת, מיפוי זיכרון ופעולות קלט/פלט מול הדיסק (I/O) הם שיכתיבו את הקצב. להלן ניתוח זמנים כן וריאלי שיאפשר לכם לכוון ציפיות ולדעת אם אתם עומדים בסד זמנים הגיוני.

11.1 שלב 1: התקנות, הגדרת הסביבה והורדת המודלים

זמן משוער: 1.5 עד 3 שעות (ברובו זמן המתנה פסיבי). **זמן עבודה פעיל:** כ-15 דקות.

מה קורה כאן: הבקשה מסוכן ה-AI להקים סביבה וירטואלית (venv), להתקין את AirLLM ולכתוב סקריפט הורדה מ-Hugging Face אורכת שניות. צוואר הבקבוק האמיתי הוא הורדת קבצי מודל עצומים (עשרות גיגה-בייטים) על גבי רוחב הפס שלכם. בנוסף, AirLLM מבצע פיצול (Sharding) של המודל לשכבות בודדות על הדיסק – משימה פיזית כבדה שאין דרך תוכניתית לעקוף.

11.2 שלב 2: הרצות, ניסויים ומדידות

זמן משוער: 3 עד 5 שעות (זמן מחשוב פיזי). **זמן עבודה פעיל:** 30 עד 45 דקות.

מה קורה כאן: הסוכן יכתוב מיד את סקריפט המדידה המדויק לחישוב TTFT וקצב TPOT. אולם בעת ייצור קריסת הבסיס (Baseline) תצטרכו להמתין עד שהמחשב ימלא את הזיכרון הפיזי ואת זיכרון ה-Swap, מה שעלול לתקוע את המערכת לכמה דקות. הרצת המודל דרך AirLLM איטית מיסודה: הספרייה משתמשת ב-mmap כדי לטעון ולפרוק כל שכבת טרנספורמר מהדיסק בנפרד, וייצור כל טוקן דורש קריאה מאסיבית מה-SSD. תצטרכו להריץ את הניסוי מספר פעמים עבור רמות קוונטיזציה שונות, ולכן פשוט המתינו בסבלנות.

טיפ

אם הדיבוג נמשך שעות – עצרו, הזינו את הודעת השגיאה חזרה לסוכן, ותנו לו לעבוד.

11.3 שלב 3: עיבוד נתונים, השוואת ביצועים וניתוח כלכלי

זמן משוער: 1 עד 1.5 שעות. **זמן עבודה פעיל:** כ-30 דקות.

מה קורה כאן: כאן מתקבל היתרון הגדול ביותר של Vibe Coding. הזינו את תוצאות המדידות ואת עלויות החומרה לסוכן, ובקשו ממנו לייצר קוד (למשל Python עם Matplotlib) המשרטט את גרף נקודת האיזון (Break-even). תנו לסוכן לבצע את חישובי החשמל והפחת, והורו לו לשקלל הנחות מודרניות של שירותי API כגון Prompt Caching. תפקידכם הנדסי טהור: להנחות, לדייק את הפרומפטים, ולוודא שהתוצאות משקפות מציאות כלכלית אמיתית.

11.4 שלב 4: חיבור וכתיבת הדוח הטכני (README)

זמן משוער: 1 עד 1.5 שעות. **זמן עבודה פעיל:** כ-1 שעה.

מה קורה כאן: ספקו לסוכן את כל הקוד, הגרפים והמספרים שאספתם, ובקשו ממנו לארוז זאת לדוח אקדמי מסודר. ודאו שהדוח אינו רק מציג מספרים אלא מסביר אותם לאור התיאוריה: האם השלב חסום בכוח חישוב (Compute-bound) או ברוחב פס זיכרון (Memory-bound)? קראו את התוצר בעין ביקורתית של מהנדסים, דרשו תיקונים, ובצעו עריכה סופית.

11.5 סיכום הערכת הזמנים

טבלה 1: הערכת זמנים מסכמת לפי שלבים

שלב	זמן משוער	זמן עבודה פעיל
התקנות והורדת מודלים	3-1.5 שעות	כ-15 דקות
הרצות, ניסויים ומדידות	5-3 שעות	כ-45 דקות
עיבוד נתונים וניתוח כלכלי	1.5-1 שעות	כ-30 דקות
כתיבת הדוח (README)	1.5-1 שעות	כ-60 דקות
סה"כ (End-to-End)	11-6.5 שעות	כ-3 שעות

זמן ההשלמה הכולל (End-to-End) הוא כ-6.5 עד 11 שעות, אך זמן העבודה האקטיבי נטו ("זמן מסך שכלתני") הוא כ-2 עד 3 שעות בלבד. אם אתם מוצאים את עצמכם שעות ארוכות מול שורות קוד, נאבקים בתצורות Python ובמשתני סביבה – סימן שאינכם מנצלים היטב את תפיסת ה-Vibe Coding. השקיעו את הזמן הקוגניטיבי בתכנון הניסוי, בניסוח הוראות מדויקות לסוכן, ובקריאה ביקורתית של התוצאות; את שאר הזמן השאירו לעבודה הפיזית של החומרה. בהצלחה!